

Prompt:

8/17/2026, 9:54:27 PM

lanjut Tahap K

Response:

8/17/2026, 9:54:28 PM

Remembering

Clarified Stage J/K trading system and QAOA portfolio design

Worked for 6s

Tahap K — Concrete Market & Numerical Specification

Sekarang kita masuk ke tahap yang menurut saya **lebih penting daripada sekadar mulai coding**.

Pada Tahap J kita sudah mempunyai:

kerangka matematis abstrak

Pada Tahap K kita akan menentukan **angka, variabel, data, horizon, constraint, encoding, dan prosedur eksperimen secara konkret** sehingga sistem tersebut dapat benar-benar diimplementasikan.

Saya akan membuat Tahap K sebagai **spesifikasi V1.0**. Artinya, kita jangan langsung membuat sistem terlalu kompleks. Kita buat sistem yang cukup kecil untuk divalidasi secara matematis, tetapi arsitekturnya sudah mampu berkembang dari crypto ke saham internasional.

K.1. Prinsip desain utama

Ada satu keputusan yang sangat penting:

Jangan langsung mencoba membuat "hedge-fund-grade system" dalam versi pertama.

Kita akan membuat:

$V1 = \text{small but mathematically rigorous}$

Kemudian:

$$V_1 \rightarrow V_2 \rightarrow V_3 \rightarrow \dots$$

Dengan demikian setiap peningkatan dapat diuji secara independen.

K.2. Universe V1

Untuk tahap pertama saya memilih **6 aset crypto** sebagai universe utama:

$$U_C = \{BTC, ETH, SOL, BNB, XRP, ADA\}$$

Alasannya bukan karena enam aset ini pasti merupakan pilihan trading terbaik, tetapi karena jumlah ini cukup untuk menghasilkan masalah kombinatorial namun masih sangat mudah diverifikasi secara exact.

Jumlah aset:

$$N = 6.$$

Kemudian kita mempunyai beberapa **factor assets** yang tidak harus masuk ke portfolio:

$$F = \{BTC, ETH, DXY, Gold, SPX, VIX\}.$$

BTC dan ETH dapat berfungsi ganda sebagai:

1. tradable asset,
 2. market factor.
-

K.3. Mengapa kita tidak langsung menggunakan 50–500 aset?

Karena kita ingin membandingkan:

Exact

melawan:

Classical

melawan:

QAOA.

Untuk $N = 6$, jika setiap aset hanya mempunyai binary selection:

$$2^6 = 64$$

portfolio.

Itu sangat kecil.

Kita dapat melakukan:

brute force

untuk menemukan global optimum secara pasti.

Ini menjadi **ground truth**.

Kemudian kita dapat meningkatkan kompleksitas.

K.4. Tahap eskalasi ukuran

Saya menyarankan:

Experiment	Assets	Binary variables	Tujuan
E1	4	4–8	Validasi matematis
E2	6	6–12	Benchmark exact
E3	10	10–20	QUBO benchmark
E4	20	20–40	Scaling
E5	50+	50+	Realistic portfolio

Dengan demikian:

$E1 \rightarrow E2 \rightarrow E3 \rightarrow E4 \rightarrow E5$

dan bukan langsung melompat ke E5.

K.5. Timeframe utama

Untuk V1 kita menggunakan:

$$\Delta t = 1 \text{ hour}$$

sebagai timeframe trading.

Mengapa?

Karena kita ingin berada di antara:

- terlalu cepat seperti HFT,
- terlalu lambat seperti daily portfolio management.

Jadi sistem kita adalah:

$$\textit{short horizon systematic trading}$$

bukan HFT.

K.6. Forecast horizon

Forecast utama:

$$H = 1h.$$

Sehingga:

$$\hat{\mu}_{i,t} = E[r_{i,t+1h} \mid \mathcal{F}_t].$$

Namun kita juga akan menghitung:

$$H \in \{1h, 4h, 24h\}$$

untuk **trade management**, bukan sebagai tiga model terpisah pada tahap pertama.

K.7. Training window

Kita menggunakan rolling window:

$$W = 30 \text{ days}$$

untuk V1.

Karena:

$$30 \times 24 = 720$$

observasi hourly.

Jadi pada waktu t :

$$D_t = \{\mathbf{r}_{t-719}, \dots, \mathbf{r}_t\}.$$

K.8. Mengapa rolling window?

Karena kita tidak ingin mengasumsikan:

$$P(\mathbf{r}_{t+1} | F_t) = P(\mathbf{r}_{t+1})$$

selamanya sama.

Pasar memiliki:

$$\text{non-stationarity.}$$

Dengan rolling window:

$$\theta_t = f(D_{t-W:t})$$

sehingga parameter dapat berubah terhadap waktu.

K.9. Data yang dibutuhkan

Untuk setiap asset:

$$OHLCV$$

yaitu:

- Open,

- High,
- Low,
- Close,
- Volume.

Selain itu:

market factors

dan kemudian, pada versi berikutnya:

order book.

K.10. Kita jangan langsung memasukkan order book ke V1

Ini keputusan desain yang penting.

Order book memiliki dimensi yang jauh lebih besar:

$$Bid_1, \dots, Bid_L$$

$$Ask_1, \dots, Ask_L$$

beserta:

$$Q_l^{bid}, \quad Q_l^{ask}.$$

Kalau kita memasukkannya sekarang, kita akan sulit mengetahui apakah perubahan performa berasal dari:

- forecasting,
- factor model,
- QUBO,
- QAOA,
- atau microstructure.

Jadi:

V1 : OHLCV + factors

dan:

$$V_2 : +order\ book.$$

K.11. Return

Gunakan:

$$r_{i,t} = \ln \frac{P_{i,t}}{P_{i,t-1}}$$

seperti yang sudah kita tetapkan.

K.12. Feature vector

Untuk setiap asset i , kita buat:

$$\mathbf{x}_{i,t} = [r_{i,t}^{(1)}, r_{i,t}^{(4)}, r_{i,t}^{(24)}, \sigma_{i,t}, v_{i,t}, \beta_{BTC,i,t}, \beta_{ETH,i,t}, \beta_{DXY,i,t}, \beta_{Gold,i,t}]^T.$$

Di sini:

$$r^{(H)} = \sum_{k=0}^{H-1} r_{t-k}.$$

K.13. Volatility

Rolling volatility:

$$\sigma_{i,t} = \sqrt{\frac{1}{W-1} \sum_{\tau=t-W+1}^t (r_{i,\tau} - \bar{r}_i)^2}$$

dengan:

$$\bar{r}_i = \frac{1}{W} \sum_{\tau=t-W+1}^t r_{i,\tau}.$$

K.14. Volume feature

Gunakan normalized volume:

$$v_{i,t} = \frac{V_{i,t}}{EMA_W(V_i)}.$$

Sehingga:

$$v_i > 1$$

berarti volume relatif tinggi.

K.15. Factor exposure

Untuk setiap asset:

$$\mathbf{f}_t = [r_{BTC,t}, r_{ETH,t}, r_{DXY,t}, r_{Gold,t}, r_{SPX,t}, r_{VIX,t}]^T.$$

Kemudian:

$$r_{i,t} = \alpha_i + \boldsymbol{\beta}_i^T \mathbf{f}_t + \epsilon_{i,t}.$$

K.16. Forecast model V1

Kita jangan langsung menggunakan neural network.

V1 menggunakan model yang sangat interpretable:

$$\hat{r}_{i,t+1} = \alpha_i + \boldsymbol{\theta}_i^T \mathbf{x}_{i,t}.$$

Secara vector:

$$\hat{\boldsymbol{\mu}}_{t+1} = \mathbf{A}_t + \boldsymbol{\Theta}_t \mathbf{X}_t.$$

Ini sengaja sederhana.

Karena kalau QAOA kemudian memberikan hasil buruk, kita dapat mengatakan:

masalahnya berasal dari optimization layer,

bukan:

mungkin neural network-nya buruk.

K.17. Regularisasi forecasting

Agar parameter tidak terlalu overfit:

$$\min_{\theta} \|y - X\theta\|_2^2 + \lambda_f \|\theta\|_2^2.$$

Ini adalah ridge regression.

Solusinya:

$$\theta^* = (X^T X + \lambda_f I)^{-1} X^T y.$$

K.18. Covariance estimator

Untuk V1:

$$\Sigma_t = \text{Cov}(\mathbf{r}_{t-W+1:t}).$$

Tetapi untuk implementasi yang lebih stabil, saya menyarankan shrinkage:

$$\Sigma_t^{shrink} = (1 - \rho)\Sigma_t + \rho D_t$$

dengan:

$$D_t = \text{diag}(\Sigma_t).$$

K.19. Mengapa shrinkage?

Karena covariance matrix empiris dapat menjadi noisy.

Terutama ketika:

$$N$$

mendekati:

$$W.$$

Shrinkage membuat:

$$\Sigma$$

lebih stabil dan positive definite.

K.20. Regime detection V1

Kita menggunakan tiga regime sederhana:

$$\text{Regime}_t \in \{Bull, Neutral, Bear\}.$$

Definisikan market return:

$$r_{M,t} = \frac{1}{N} \sum_i r_{i,t}.$$

Kemudian:

$$z_t = \frac{EMA(r_M) - EMA_{long}(r_M)}{\sigma_M}.$$

Aturan sederhana:

$$\text{Regime}_t = \begin{cases} Bull, & z_t > \delta \\ Neutral, & |z_t| \leq \delta \\ Bear, & z_t < -\delta. \end{cases}$$

K.21. Tetapi regime jangan langsung menjadi constraint

Ini penting.

Regime pada V1 digunakan sebagai:

$$\boxed{\text{feature}}$$

dan bukan:

hard constraint.

Jadi model dapat belajar bahwa:

Bear

mungkin mengurangi expected return, tetapi tidak otomatis memaksa:

$$w = 0.$$

K.22. Universe selection

Setiap periode:

t

kita ranking asset berdasarkan:

$$Score_i = \frac{\hat{\mu}_i}{\hat{\sigma}_i}.$$

Kemudian pilih:

$$K = 3.$$

Sehingga:

$$\boxed{\sum_i x_i \leq 3.}$$

K.23. Mengapa hanya tiga?

Karena kita ingin membuat:

selection + allocation

menjadi masalah combinatorial yang nyata.

Jika semua asset boleh dipilih:

$$x_i = 1$$

untuk semuanya, masalah selection menjadi kurang menarik.

K.24. Weight discretization

Gunakan step:

$$\delta_w = 0.05.$$

Maka weight:

$$w_i \in \{0, 0.05, 0.10, \dots, 0.50\}.$$

Maximum per asset:

$$w_i^{max} = 0.50.$$

K.25. Binary encoding

Karena:

$$0.50/0.05 = 10,$$

kita membutuhkan representasi integer:

$$q_i \in \{0, \dots, 10\}.$$

Binary encoding:

$$q_i = b_{i0} + 2b_{i1} + 4b_{i2} + 8b_{i3}.$$

Kemudian:

$$w_i = 0.05q_i.$$

K.26. Ada masalah dengan angka 8

Karena:

$$q_i$$

dengan empat bit dapat mencapai:

$$15.$$

Sedangkan maksimum kita hanya:

$$10.$$

Maka konfigurasi:

$$q_i > 10$$

harus dibuat infeasible.

Kita dapat menambahkan:

$$P_{range}.$$

Ini contoh kecil tetapi penting dari desain QUBO.

K.27. Budget constraint

Kita ingin:

$$\sum_i w_i = 1.$$

Penalty:

$$P_B = A_B \left(\sum_i w_i - 1 \right)^2.$$

K.28. Cardinality constraint

$$\sum_i x_i \leq 3.$$

Penalty:

$$P_K = A_K \max \left(0, \sum_i x_i - 3 \right)^2.$$

Untuk QUBO murni nanti kita ubah menggunakan slack variable.

K.29. Selection-size coupling

Kita harus memastikan:

$$x_i = 0 \Rightarrow w_i = 0.$$

Gunakan:

$$P_{coupling} = A_C \sum_i w_i (1 - x_i).$$

Jika:

$$x_i = 0$$

tetapi:

$$w_i > 0,$$

maka ada penalty.

K.30. Financial objective V1

Kita gunakan:

$$C(\mathbf{w}) = -\boldsymbol{\mu}^T \mathbf{w} + \lambda \mathbf{w}^T \Sigma \mathbf{w} + \kappa \|\mathbf{w} - \mathbf{w}_{prev}\|_1.$$

Tiga komponen:

$$-\boldsymbol{\mu}^T \mathbf{w}$$

= reward expected return,

$$\lambda \mathbf{w}^T \Sigma \mathbf{w}$$

= risk,

dan:

$$\kappa \|\mathbf{w} - \mathbf{w}_{prev}\|_1$$

= turnover penalty.

K.31. Mengapa turnover penting?

Tanpa turnover penalty:

$$\mathbf{w}_t^*$$

dapat berubah drastis setiap jam.

Misalnya:

$$\mathbf{w}_t = [0.5, 0.3, 0.2]$$

kemudian satu jam kemudian:

$$\mathbf{w}_{t+1} = [0, 0.1, 0.9].$$

Secara matematis mungkin optimal.

Secara trading:

sangat mahal

karena banyak transaksi.

K.32. Transaction cost

Kita gunakan model awal:

$$TC_t = c_{linear} \sum_i |w_{i,t} - w_{i,t-1}|$$

dengan:

$$c_{linear}$$

sebagai biaya efektif.

Kemudian pada versi order-book:

$$TC = Fee + Spread + Slippage + MarketImpact.$$

K.33. Objective lengkap

Jadi:

$$C_t(\mathbf{w}, \mathbf{x}) = -\boldsymbol{\mu}_t^T \mathbf{w} + \lambda \mathbf{w}^T \Sigma_t \mathbf{w} + \kappa \|\mathbf{w} - \mathbf{w}_{t-1}\|_1 + A_B (\mathbf{1}^T \mathbf{w} - 1)^2 + A_K P_K(\mathbf{x}) + A_C \sum_i w_i (1 - x_i).$$

K.34. Risk aversion

Parameter:

$$\lambda$$

mengontrol:

$$Return \leftrightarrow Risk.$$

Jika:

$$\lambda \rightarrow 0,$$

model semakin agresif.

Jika:

$$\lambda \rightarrow \infty,$$

model semakin konservatif.

Kita jangan menentukan λ dengan melihat hasil backtest kemudian.

Itu akan menjadi:

$$\boxed{\text{data leakage}}.$$

K.35. Parameter calibration

Kita menggunakan training period untuk menentukan:

$$\lambda, \kappa, A_B, A_K, A_C.$$

Kemudian parameter dibekukan selama test period.

K.36. Penentuan penalty coefficient

Penalty harus lebih besar daripada keuntungan yang mungkin diperoleh dari melanggar constraint.

Secara konseptual:

$$\boxed{A_{\text{constraint}} > \max |\Delta C_{\text{financial}}|}.$$

Sehingga:

constraint violation

tidak pernah lebih menguntungkan daripada solusi feasible.

K.37. QUBO

Setelah semua binary substitution:

$$w_i = w_i(\mathbf{b}_i)$$

dan:

$$x_i = b_{x_i},$$

kita akan mendapatkan:

$$C(\mathbf{z}) = \mathbf{z}^T Q \mathbf{z} + \mathbf{c}^T \mathbf{z} + c.$$

Dengan:

$$\mathbf{z} \in \{0, 1\}^M.$$

K.38. Jumlah qubit

Untuk enam asset:

Selection

6 qubits.

Weight

4 qubits per asset:

$$6 \times 4 = 24.$$

Total:

$$M = 30$$

sebelum slack variables.

Ini sudah cukup besar untuk menunjukkan mengapa masalah ini menarik secara quantum optimization.

K.39. Tetapi jangan langsung menjalankan QAOA 30 qubit

Ini sangat penting.

Untuk validasi:

$$N = 4$$

terlebih dahulu.

Misalnya:

$$M \approx 20.$$

Kemudian:

$$N = 6$$

dan seterusnya.

K.40. Classical exact solver

Untuk:

$$M \leq 25$$

kita dapat menggunakan brute force jika memungkinkan.

Untuk setiap:

$$z \in \{0, 1\}^M$$

hitung:

$$C(z).$$

Kemudian:

$$z^* = \arg \min_z C(z).$$



Ini adalah ground truth.

K.41. Classical optimization baseline

Kita tidak cukup hanya membandingkan:

QAOA

dengan brute force.

Kita perlu baseline realistis:

MILP/QUBOclassicalsolver

dan:

simulated annealing.

Untuk continuous version:

SLSQP

atau optimizer constrained classical yang setara.

K.42. Jadi kita mempunyai empat baseline

Exact

Classical Continuous

Classical Combinatorial

QAOA

Ini jauh lebih kuat daripada sekadar:

"Saya menjalankan QAOA."

K.43. QAOA V1

Cost Hamiltonian:

$$H_C = \sum_i h_i Z_i + \sum_{i < j} J_{ij} Z_i Z_j.$$

Mixer:

$$H_M = \sum_i X_i.$$

Depth awal:

$$p = 1.$$

Kemudian:

$$p = 2, 3$$

sebagai eksperimen scaling.

K.44. Mengapa mulai dari $p = 1$?

Karena:

$$p \uparrow$$

berarti:

- circuit semakin dalam,
- parameter semakin banyak,
- optimization semakin sulit,
- noise semakin besar jika hardware digunakan.

Kita ingin mengetahui apakah:

$$p = 1$$

sudah memberikan solusi yang berguna.

K.45. QAOA parameter optimizer

V1:

$$\text{COBYLA}$$

atau optimizer derivative-free sejenis.

Kemudian dibandingkan dengan:

$$\text{SPSA}.$$

Karena kita memang sebelumnya sudah mempelajari SPSA dan parameter-shift.

K.46. Objective QAOA

$$E(\gamma, \beta) = \langle \psi(\gamma, \beta) | H_C | \psi(\gamma, \beta) \rangle.$$

Cari:

$$(\gamma^*, \beta^*) = \arg \min_{\gamma, \beta} E.$$

K.47. Quality metric QAOA

Jangan hanya mengukur apakah QAOA menghasilkan profit.

Pertama:

$$\text{Approximation Ratio}$$

atau cost gap.

Untuk minimization:

$$Gap = \frac{C_{QAOA} - C^*}{|C^*|}.$$

Semakin dekat:

$$Gap \rightarrow 0,$$

semakin baik.

K.48. Probability of sampling optimum

Karena QAOA menghasilkan distribusi:

$$p(z),$$

kita dapat mengukur:

$$P_{opt} = p(z^*).$$

Ini sangat penting.

QAOA tidak harus selalu menghasilkan optimum dalam setiap measurement.

Yang kita inginkan adalah:

$$P_{opt}$$

cukup tinggi.

K.49. Feasibility probability

Kita juga ukur:

$$P_{feasible} = \sum_{z \in F} p(z)$$

dengan:

$$F$$

adalah set solusi feasible.

Ini bahkan bisa menjadi lebih penting daripada:

$$P_{opt}.$$

K.50. Mengapa?

Bayangkan:

$$P_{opt} = 0.05$$

tetapi:

$$P_{feasible} = 0.95.$$

Ini masih cukup bagus.

Tetapi jika:

$$P_{opt} = 0.20$$

dan:

$$P_{feasible} = 0.25,$$

maka sebagian besar distribution QAOA menghasilkan portfolio yang melanggar constraint.

K.51. Trade construction

Setelah portfolio:

$$w^*,$$

kita menentukan entry signal.

Jangan langsung market order.

Kita definisikan:

$$Signal_i = f(\mu_i, \sigma_i, Regime, Liquidity).$$

K.52. Entry threshold

Misalnya:

$$\hat{\mu}_i > \theta_\mu \sigma_i.$$

Dengan kata lain:

expected return harus cukup besar dibanding uncertainty.

Jika:

$$\frac{\hat{\mu}_i}{\sigma_i} \leq \theta_\mu,$$

tidak ada trade.

K.53. Limit order

Jika signal valid:

$$Order_i = Limit(P_{entry}).$$

Order **tidak langsung dieksekusi**.

Ini sesuai dengan filosofi yang kita bangun sebelumnya.

K.54. Order expiry

Limit order tidak boleh hidup selamanya.

Definisikan:

$$T_{expiry} = t + H_{entry}.$$

Misalnya:

$$H_{entry} = 1h.$$

Jika tidak fill:

Cancel.

K.55. Fill model V1

Karena kita belum menggunakan tick-level order book:

Fill

ditentukan berdasarkan OHLC.

Misalnya untuk long:

$$P_{low,t} \leq P_{entry}$$

maka order dianggap fill.

Tetapi ini memiliki masalah:

Apakah harga menyentuh entry sebelum SL/TP?

Karena itu kita memerlukan aturan konservatif.

K.56. Ambiguity rule

Jika dalam satu candle:

$$Low \leq SL$$

dan:

$$High \geq TP,$$

kita **tidak boleh memilih outcome yang menguntungkan.**

Gunakan worst-case ordering:

SL dianggap terjadi lebih dahulu.

Ini menghindari look-ahead bias.

K.57. Initial stop-loss

Untuk long:

$$SL = P_{entry} - k_{SL}\sigma_i.$$

V1:

$$k_{SL} = 2.$$

Jadi:

$$d_{SL} = 2\sigma_i.$$

K.58. Risk per trade

Misalnya:

$$R_{trade} = \$1.$$

Maka:

$$Q_i = \frac{1}{|P_{entry} - P_{SL}|}.$$

Tetapi ini belum memperhitungkan fee/slippage.

Nanti:

$$Q_i = \frac{R_{trade}}{d_{SL} + Cost_{roundtrip}}.$$

K.59. Portfolio-level risk

Jika ada tiga posisi:

$$R_1 = R_2 = R_3 = \$1,$$

maka bukan berarti portfolio risk otomatis:

\$3.

Karena posisi dapat correlated.

Portfolio risk:

$$\sigma_p = \sqrt{\mathbf{w}^T \Sigma \mathbf{w}}.$$

Ini alasan factor/covariance model tetap diperlukan pada trade-level.

K.60. Maximum aggregate risk

V1:

$$R_{portfolio,max} = 3R_{trade}.$$

Tetapi kemudian kita akan menggunakan correlation-adjusted risk.

K.61. Take profit V1

Gunakan:

$$TP = P_{entry} + k_{TP} \sigma_i.$$

Dengan:

$$k_{TP} = 3.$$

Sehingga initial:

$$RR = \frac{3\sigma}{2\sigma} = 1.5.$$

Ini **bukan angka sakral**. Nanti diuji melalui walk-forward training.

K.62. Dynamic SL

Jika:

$$P_{nL} > 0$$

dan:

$$\frac{P_t - P_{entry}}{\sigma_i} > k_{BE},$$

kita dapat memindahkan:

$$SL \rightarrow BEP.$$

Misalnya:

$$k_{BE} = 1.$$

K.63. Trailing stop

Setelah BEP:

$$SL_t = \max(SL_{t-1}, P_t - k_{trail}\sigma_t).$$

Ini menjamin:

$$SL_t \geq SL_{t-1}.$$

K.64. Dynamic TP

Jika continuation value:

$$V_{hold} > V_{close},$$

maka:

$$TP$$

dapat diperbesar.

Misalnya:

$$k_{TP,new} = \min(k_{TP,old} + \Delta k, k_{TP,max}).$$

V1:

$$\Delta k = 0.5$$

dan:

$$k_{TP,max} = 5.$$

K.65. Exit decision

Setiap satu jam:

EvaluatePosition()

dengan:

μ_t ,
 σ_t ,
 $Regime_t$,
 V_{hold} ,
 V_{close} ,
 SL_t ,
 TP_t .

K.66. Decision rule

Secara konseptual:

$$Decision = \begin{cases} Close, & V_{hold} < V_{close} \\ MoveSL, & V_{hold} > V_{close}, PnL > 0 \\ MoveTP, & V_{hold} \gg V_{close} \\ Hold, & otherwise. \end{cases}$$

Ini akan kita buat lebih formal ketika masuk ke tahap implementasi trade-management.

K.67. Maximum holding period

Sebuah trade tidak boleh bertahan selamanya.

Definisikan:

$$T_{max} = 24h.$$

Jika:

$$t - t_{entry} > 24h,$$

maka:

Close.

K.68. Ini penting untuk menghindari hidden risk

Tanpa:

$$T_{max},$$

sistem short-term dapat berubah menjadi:

buy and hope.

Padahal model kita seharusnya:

short horizon.

K.69. Walk-forward backtesting

Sekarang bagian yang sangat penting.

Tidak boleh:

$$Train_{2021-2024} \rightarrow Test_{2021-2024}.$$

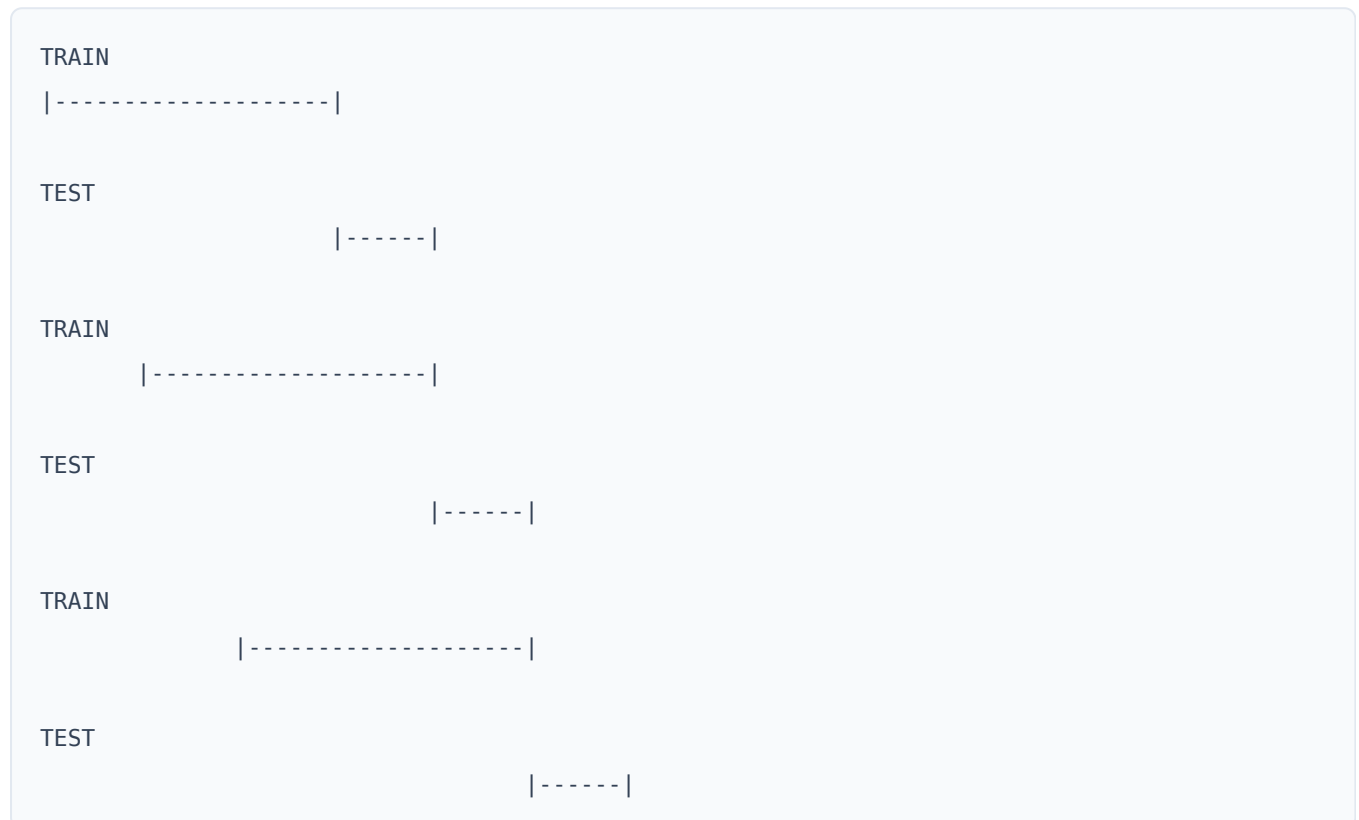
Itu bukan backtest yang valid.

Kita gunakan:

Walk Forward.

K.70. Struktur walk-forward

Misalnya:



Setiap test period benar-benar out-of-sample.

K.71. Contoh konkret

Misalnya:

Train = 30 days

dan:

Test = 7 days.

Kemudian:

$$Train_2 = Train_1 + 7days.$$

K.72. Rolling versus expanding

Untuk sistem kita:

rolling window

lebih sesuai.

Jadi:

$$D_t = [t - W, t].$$

Bukan:

$$D_t = [0, t].$$

K.73. Kenapa?

Karena market regime berubah.

Data sepuluh tahun lalu mungkin mempunyai relevansi lebih rendah daripada:

30 hari terakhir.

Tetapi ini adalah hipotesis yang harus diuji, bukan asumsi final.

K.74. Backtest pipeline

Pada setiap waktu t :

1. *Load historical data*
2. *Build features*
3. *Estimate factors*
4. *Forecast returns*
5. *Estimate covariance*
6. *Generate QUBO*
7. *Solve QUBO*
8. *Generate target allocation*
9. *Generate orders*
10. *Simulate execution*
11. *Manage positions*
12. *Record PnL.*

Kemudian:

$$t \rightarrow t + 1.$$

K.75. Yang harus disimpan setiap timestamp

Database/backtest record minimal:

$$Record_t = \{timestamp, state, forecast, covariance, qubo, solution, orders, fills, positions, SL, TP, PnL\}.$$

Ini akan sangat berguna untuk debugging.

K.76. Performance metrics

Kita tidak boleh hanya menggunakan:

$$Total\ PnL.$$

Minimal:

Return

$$R_{total}.$$

Sharpe

$$Sharpe = \frac{E[R_p - R_f]}{\sigma_p}$$

Sortino

$$Sortino = \frac{E[R_p - R_f]}{\sigma_{down}}$$

Maximum Drawdown

$$MDD = \max_t \left(\frac{Peak_t - Equity_t}{Peak_t} \right).$$

K.77. Trade metrics

Kita juga ukur:

$$WinRate = \frac{N_{win}}{N_{trade}}.$$

Profit factor:

$$PF = \frac{GrossProfit}{|GrossLoss|}.$$

Average win:

$$\bar{W}.$$

Average loss:

$$\bar{L}.$$

Expectancy:

$$E = p_w \bar{W} - (1 - p_w) |\bar{L}|.$$

K.78. QAOA-specific metrics

Selain financial metrics:

<i>Cost Gap</i>
<i>Approximation Quality</i>
P_{opt}
$P_{feasible}$
<i>Circuit Depth</i>
<i>Number of Qubits</i>
<i>Classical Runtime</i>
<i>QAOA Runtime</i>

K.79. Ini menghasilkan dua evaluation layers

Layer 1 — Mathematical

<i>QAOA vs Classical</i>

berdasarkan:

$C(z)$.

Layer 2 — Financial

<i>Trading Strategy</i>

berdasarkan:

$PnL, Sharpe, MDD, PF, \dots$

Kita **harus memisahkan kedua layer ini**.

K.80. Eksperimen inti

Saya menyarankan eksperimen utama berikut.

Experiment A

Exact

vs

QAOA.

Tujuan:

Apakah QAOA mampu menemukan solusi dekat optimum?

Experiment B

Classical

vs

QAOA.

Tujuan:

Apakah QAOA kompetitif terhadap classical combinatorial optimizer?

Experiment C

Trading tanpa QAOA:

Forecast → Classical → Trading.

Bandingkan dengan:

Forecast → QAOA → Trading.

K.81. Experiment D — no-factor model

Gunakan:

$$\hat{\mu}_i = f(\text{asset}_i)$$

tanpa:

BTC, ETH, DXY, Gold, ...

Kemudian bandingkan dengan:

Factor – aware.

Ini menguji apakah market factors benar-benar menambah informasi.

K.82. Experiment E — no dynamic management

Bandingkan:

Static

SL, TP

tidak berubah.

melawan:

Dynamic

SL_t, TP_t

berubah berdasarkan state.

Ini menguji ide yang kita diskusikan sejak awal.

K.83. Experiment F — transaction cost sensitivity

Jalankan:

TC = 0

kemudian:

$$TC = low$$

kemudian:

$$TC = medium$$

kemudian:

$$TC = high.$$

Jika strategy hanya profitable ketika:

$$TC = 0,$$

maka secara praktis strategy tersebut tidak menarik.

K.84. Experiment G — regime robustness

Evaluasi terpisah:

Bull, Neutral, Bear.

Kita ingin mengetahui apakah strategy hanya bekerja dalam satu regime.

K.85. Experiment H — quantum depth

Bandingkan:

$$p = 1$$

$$p = 2$$

$$p = 3.$$

Kemudian:

$$Gap(p).$$

Kita ingin mengetahui:

$$\frac{dGap}{dp}.$$

Apakah depth tambahan benar-benar memberikan improvement?

K.86. Experiment I — scaling

Naikkan:

$$N = 4, 6, 10, 20.$$

Catat:

$$M(N)$$

dan:

$$Runtime(N).$$

Kemudian:

$Scaling\ Curve$

menjadi salah satu hasil penelitian yang sangat menarik.

K.87. Experiment J — stochastic robustness

Karena QAOA menghasilkan probabilitas:

$$p(z),$$

jalankan measurement berulang:

$$Shots = \{100, 500, 1000, 5000\}.$$

Kemudian ukur:

$$P_{opt}(Shots).$$

K.88. Data split

Kita harus mempunyai tiga bagian:

$Train$

$Validation$

$Test.$

Test set **tidak boleh digunakan untuk memilih parameter.**

K.89. Parameter yang harus dibekukan

Setelah validation selesai:

$$\Theta^* = \{W, \lambda, \kappa, \delta_w, K, k_{SL}, k_{TP}, T_{max}, p\}$$

dibekukan.

Baru kemudian:

$Test.$

K.90. Anti-lookahead rule

Ini akan menjadi salah satu aturan paling keras dalam implementasi.

Pada timestamp t :

$$\text{model hanya boleh mengetahui informasi } \leq t.$$

Tidak boleh menggunakan:

$$P_{t+1}$$

untuk menentukan:

$$A_t.$$

K.91. Bahkan covariance harus lagged correctly

Jika:

$$\Sigma_t$$

digunakan untuk membuat portfolio t , maka:

$$\Sigma_t$$

harus dihitung dari:

$$r_{\leq t}.$$

Bukan:

$$r_{t+1}.$$

Ini terlihat sepele tetapi merupakan salah satu sumber terbesar backtest leakage.

K.92. Execution lag

Kita juga jangan mengasumsikan:

$$Signal_t \rightarrow Fill_t$$

secara instan.

Minimal:

$$Signal_t \rightarrow Order_t \rightarrow Fill_{t+\Delta}.$$

Untuk V1:

$$\Delta = 1 \text{ candle.}$$

K.93. Mengapa?

Karena jika:

$$P_t$$

digunakan untuk menghasilkan signal dan kita langsung menganggap entry pada:

$$P_t,$$

kita berpotensi mendapatkan harga yang tidak benar-benar tersedia setelah signal dihitung.

K.94. Struktur final V1

Jadi sistem V1 kita:

Data = OHLCV + Factors

Timeframe = 1h

Forecast = Ridge

Window = 30d

Covariance = Shrinkage

Regime = 3 - state

Universe = 6 assets

Selection = $K \leq 3$

Weight = 5% increments

MaxWeight = 50%

Risk = Covariance - based

Optimizer = QUBO

Quantum = QAOA

Depth = $p = 1, 2, 3$

SL = 2σ

TP = 3σ

Risk/trade = \$1

MaxHolding = 24h

Reoptimization = 1h

Backtest = Walk - forward

K.95. Arsitektur software

Sekarang spesifikasi matematis ini dapat diterjemahkan menjadi modul:

```
trade_system/
|
├─ data/
|   ├── downloader
|   ├── cleaner
|   ├── resampler
|   └─ factor_data
|
├─ features/
|   ├── returns
|   ├── volatility
|   ├── momentum
|   └─ factor_exposure
|
├─ forecasting/
|   ├── ridge
|   └─ prediction
|
├─ risk/
|   ├── covariance
|   ├── shrinkage
|   └─ portfolio_risk
|
├─ optimization/
|   ├── classical
|   ├── exact
|   ├── qubo
|   ├── ising
|   └─ qaoa
|
├─ execution/
|   ├── order_generator
|   ├── fill_model
|   └─ transaction_cost
|
├─ position/
|   ├── stop_loss
|   ├── take_profit
|   ├── trailing
|   └─ position_manager
|
└─ backtest/
```

```
|   ├── engine
|   ├── walk_forward
|   └── metrics
|
└── experiments/
    ├── exact_vs_qaoa
    ├── factor_ablation
    ├── cost_sensitivity
    ├── qaoa_depth
    └── scaling
```

K.96. Dan di sinilah proyek kita menjadi sangat menarik

Perhatikan bahwa **QAOA** hanya membutuhkan sebagian kecil dari keseluruhan sistem.

Strukturnya:

Econometrics → Financial Mathematics → Combinatorial Optimization → Quantum Computing → Market Micro

Jadi proyek ini bukan sekadar:

"Saya menggunakan quantum computer untuk trading."

Lebih tepat:

Saya membangun dynamic factor-aware portfolio/trading optimization problem yang kemudian diformulasikan sebagai QUBO/Ising dan diselesaikan menggunakan QAOA, dengan evaluasi terhadap classical optimization dan realistic execution constraints.

Itu jauh lebih kuat secara akademik.

K.97. Satu hal yang perlu kita luruskan sejak sekarang

Saya ingin memisahkan dua sistem:

Sistem penelitian

QAOA Trading Research Engine

Tujuannya:

mengukur apakah QAOA berguna.

Sistem live trading

Production Trading Engine

Tujuannya:

menghasilkan keputusan trading secara aman.

Jangan mencampurkan keduanya.

Production engine baru boleh dibuat setelah:

Backtest → Paper Trading → Stress Test → Live Small Capital.

K.98. Roadmap setelah Tahap K

Sekarang seluruh desain abstrak kita sudah cukup konkret.

Urutan selanjutnya seharusnya:

$K \rightarrow L \rightarrow M \rightarrow N \rightarrow O$

dengan:

Tahap L — Data Engineering

Kita menentukan secara konkret:

DataSource → Schema → Timestamp → Cleaning → Alignment → Storage.

Termasuk masalah yang sangat penting seperti:

- timezone,
- missing candle,
- survivorship bias,
- delisted assets,
- corporate action untuk saham,

- perbedaan jam perdagangan saham vs crypto,
- DXY/Gold/SPX yang tidak trading 24/7,
- dan bagaimana factor data disejajarkan dengan hourly crypto data.

Tahap M — Forecasting & Factor Engine

Kita turunkan secara lengkap:

$$X \rightarrow \hat{\mu}$$

dan:

$$X \rightarrow B, \Sigma, Regime.$$

Tahap N — QUBO/Ising Engine

Ini akan menjadi salah satu bagian paling matematis:

$$Financial\ Objective \rightarrow Binary\ Encoding \rightarrow QUBO \rightarrow Ising \rightarrow Hamiltonian.$$

Kita bahkan dapat mengambil **contoh numerik 4 aset** dan menurunkan seluruh elemen:

$$Q, \quad h_i, \quad J_{ij}$$

secara eksplisit sampai menjadi:

$$\hat{H}_C.$$

Tahap O — QAOA + Classical Benchmark

Baru kita implementasikan:

$$Exact \leftrightarrow Classical \leftrightarrow QAOA.$$

Dan kita ukur:

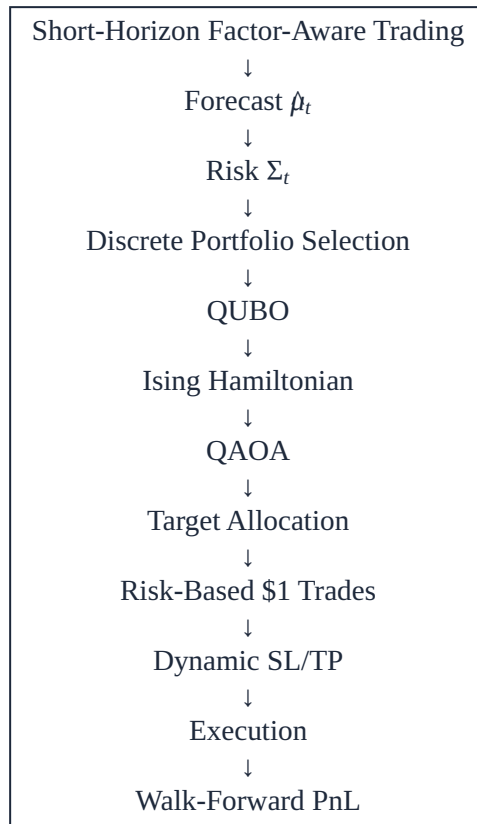
$$Gap, \quad P_{opt}, \quad P_{feasible}, \quad Runtime.$$

Posisi kita sekarang

Dengan Tahap A–K, kita sudah berpindah dari sekadar ide:

"Bagaimana kalau kita menggunakan QAOA untuk trading?"

menjadi sebuah objek penelitian yang jauh lebih terdefinisi:



Dan yang menurut saya paling penting: **arsitektur ini tidak terkunci pada crypto**. Ketika kita pindah ke saham internasional, bagian optimization engine pada dasarnya tetap sama. Yang berubah terutama adalah universe, market-calendar handling, factor set, liquidity model, dan execution model. Faktor seperti **Gold dan USD/DXY** dapat tetap menjadi latent/common drivers, sementara factor equity seperti SPX/VIX dapat ditambahkan.

Tahap berikutnya yang paling tepat adalah Tahap L — Data Engineering, karena sebelum kita menurunkan QUBO secara numerik, kita harus memastikan bahwa $\hat{\mu}_t$, Σ_t , dan seluruh factor exposure yang masuk ke QUBO benar-benar berasal dari data yang secara temporal konsisten dan bebas look-ahead. Ini justru salah satu bagian yang paling sering membuat backtest kuantitatif terlihat bagus padahal salah.